MULTITASKING - CAVEATS

If all tasks in a concurrent program are independent, then the only control that is needed is the allocation of the processor to the tasks (**scheduling**).

However, if tasks share resources (memory regions or peripherals) then an adequate control of resource sharing must be performed to guarantee **exclusive access to shared resources** and to avoid **deadlocks** and **priority inversion**.

MULTITASKING - CAVEATS

- Shared resources
  - restrictions to concurrent access;
  - order of accesses among tasks is critical and should not cause errors.
- Allocation of shared resources must consider task switching so that resources are not blocked due to suspended tasks.
- Difficult to debug concurrent programs.

CONCURRENT COMPUTATIONAL TASKS - ISSUES

The following issues must be dealt with in an multitask environment:

- Communication among tasks
- Synchronization among tasks
- Resource sharing and Mutual Exclusion
- Task scheduling

THE MUTUAL EXCLUSION PROBLEM

In a concurrent implementation, shared resources (data or devices) must be protected from simultaneous access to the so-called Critical Region.

Topics:

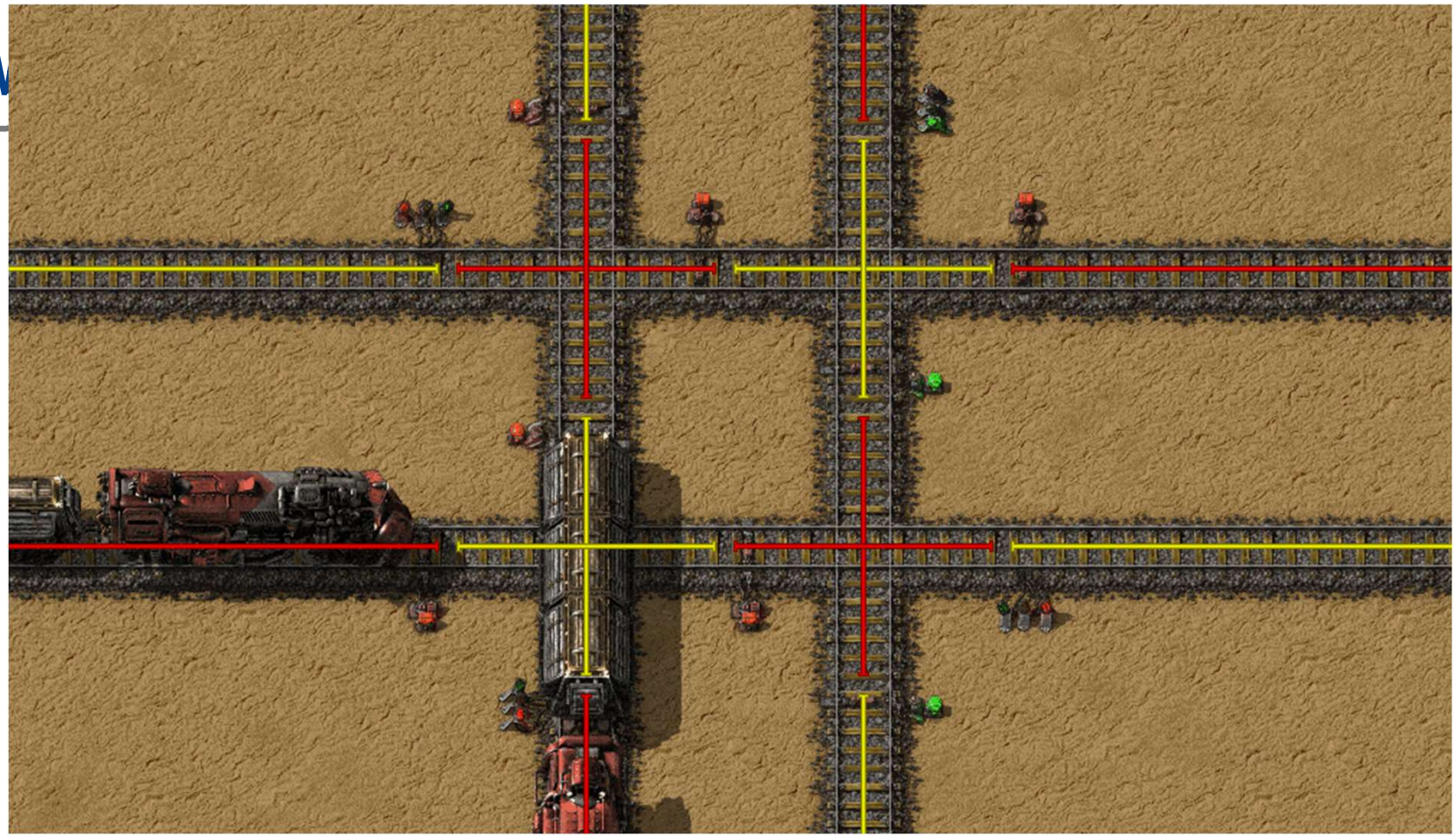
- Shared Resources and Critical Regions
- The Mutual Exclusion Problem
- Solutions for shared-memory architectures
- Solutions for networked architectures

RESOURCE SHARING AND MUTUAL EXCLUSION





W



WHAT IS A SHARED RESOURCE (COMPUTING)

In concurrent programming, shared resources are:

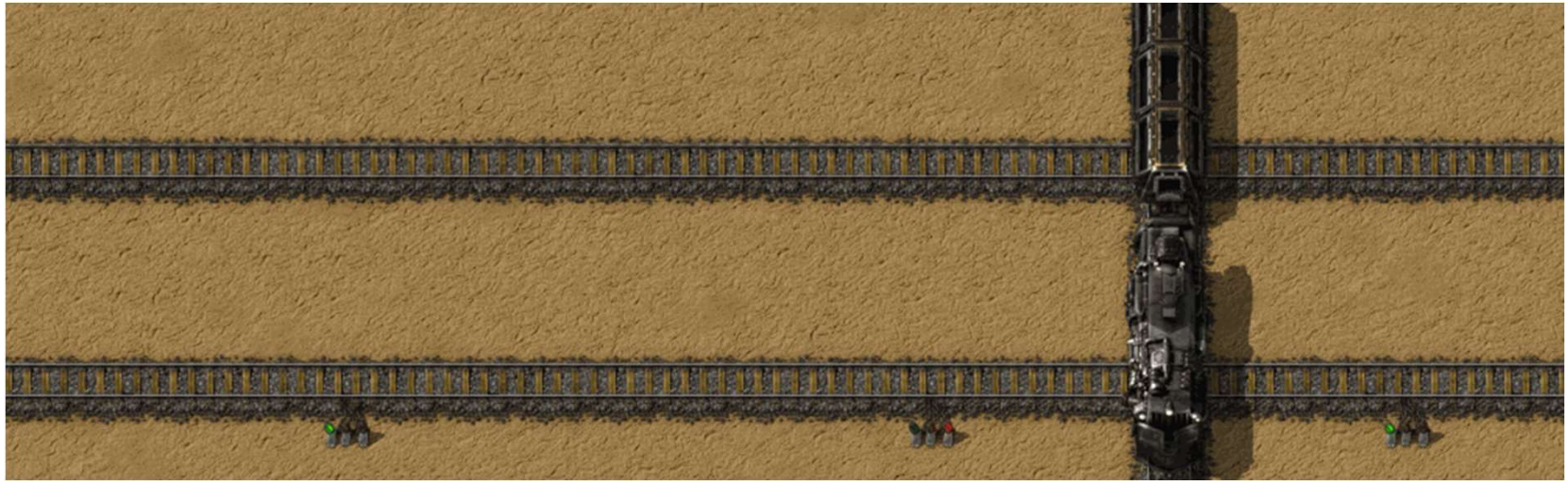
- data object
- devices
- (generalizing) addressing regions

that are shared among tasks.

Shared resources usually have restrictions concerning concurrent access, hence, the Mutual Exclusion requirement.

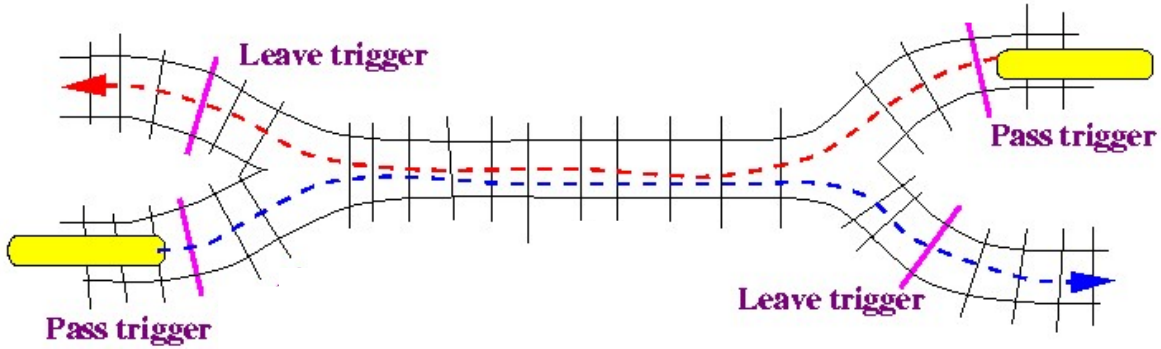
MUTUAL EXCLUSION

Example of a shared resource with mutual exclusion requirement:



MUTUAL EXCLUSION

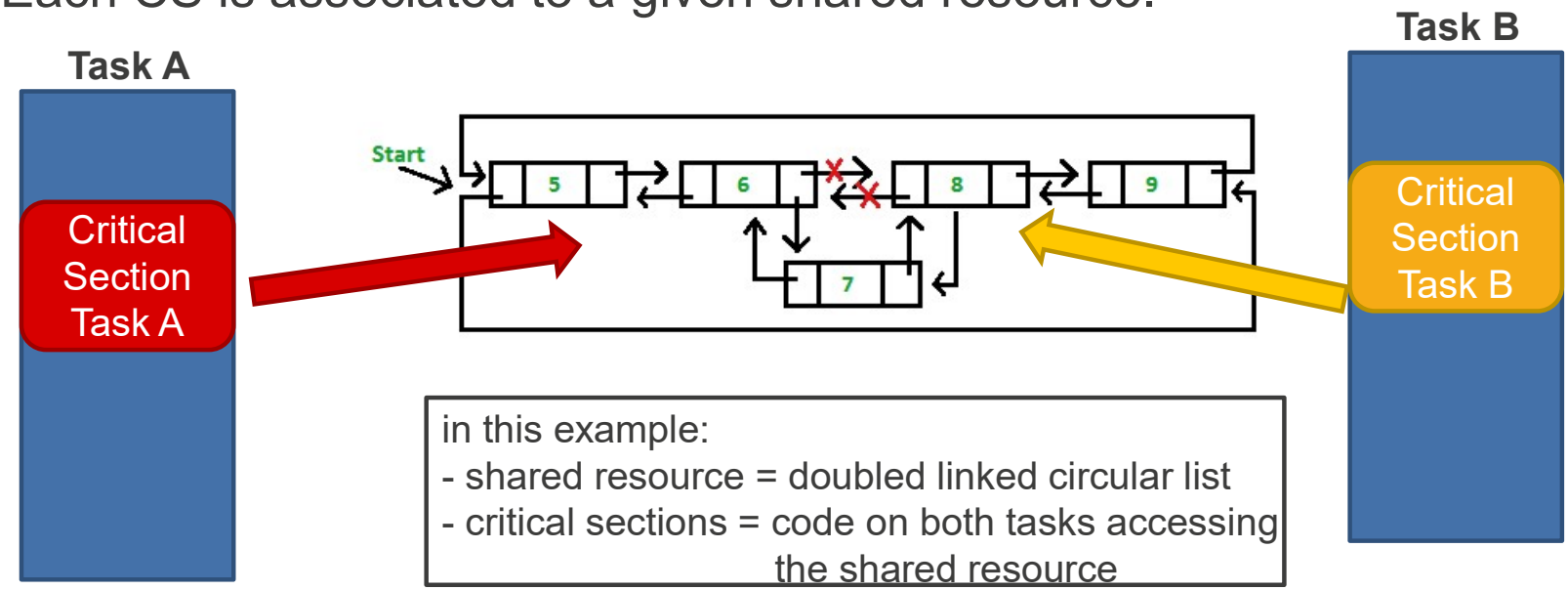
A resource, with mutual exclusion requirements, shared among N tasks may be used by a single task at any given time. Hence, if while task A is using this resource task B is willing to use it as well, then task B must wait for task A to complete its usage of the resource.



CRITICAL SECTION (CS)

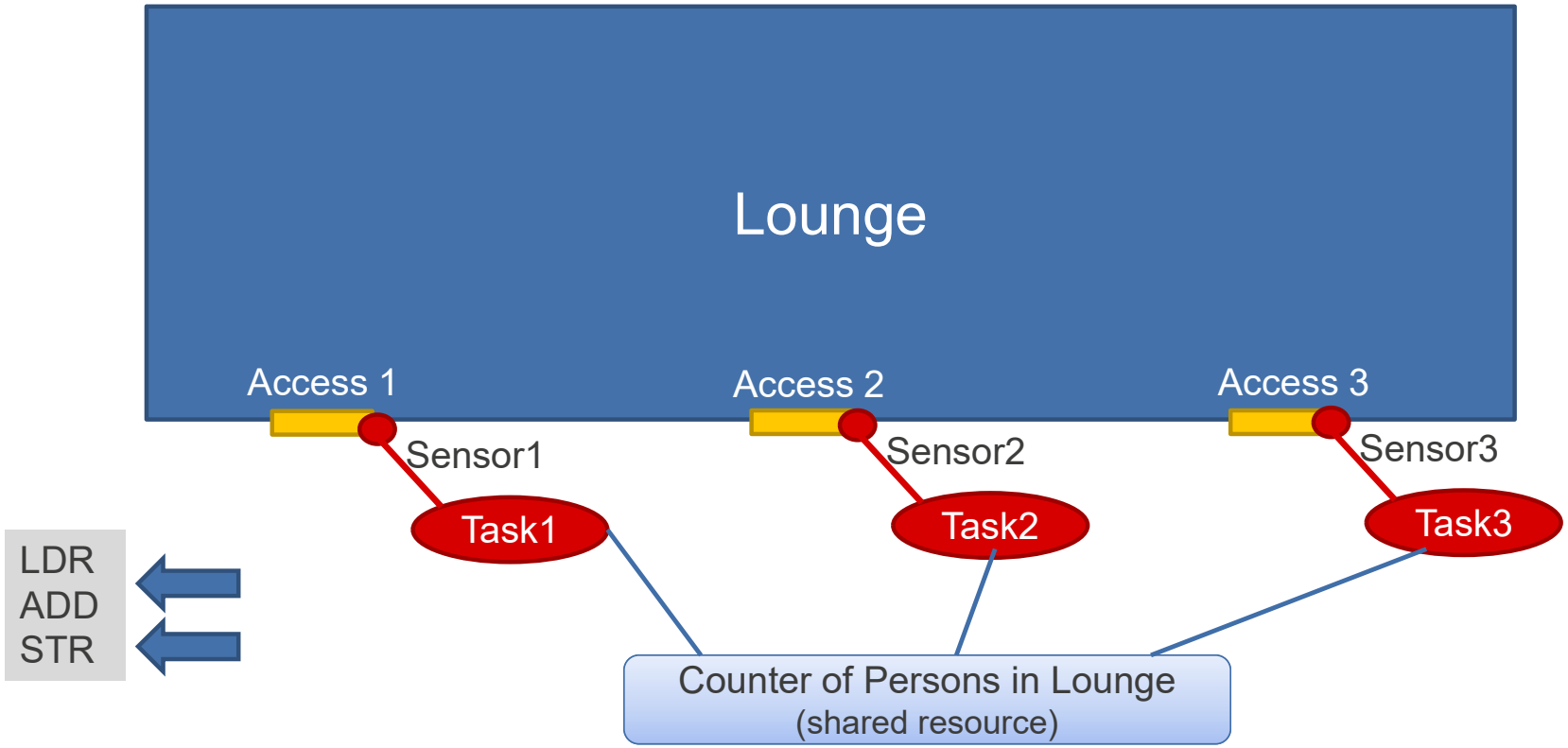
The code segment of a task where the shared resource is accessed.

Each CS is associated to a given shared resource.



in this example:
- shared resource = doubled linked circular list
- critical sections = code on both tasks accessing the shared resource

EXAMPLE -
INCORRECT CONCURRENT ACCESS TO SHARED RESOURCE



THE MUTUAL EXCLUSION PROBLEM

Requirements:

- When a task is accessing shared resources, thus, executing a Critical Section, no other task may be in a Critical Section accessing any of those resources.
- A tasks should not be delayed access to its Critical Section when no other task is using the shared resource.
- The execution time of a Critical Section is finite.
- No assumption should be made concerning the relative speed or frequency of access to the shared resource.
- Solution to the Mutual Exclusion problem must avoid:
  - Deadlock - all tasks attempting to use the shared resource are blocked;
  - Livelock - tasks are alive but unable to proceed;
  - Starvation - a task is overlooked indefinitely;
  - the order of resource access cannot be restricted (e.g. “taking turns”).

THE MUTUAL EXCLUSION PROBLEM - FORMALIZATION

As stated by Lamport, the mutual exclusion problem is formally stated by these requirements:

1. A task i is a sequence of alternating Non-critical (NCS) and critical sections (CS). Each of these are terminating operation executions.

$$NCS_i^{[1]} \longrightarrow CS_i^{[1]} \longrightarrow NCS_i^{[2]} \longrightarrow CS_i^{[2]} \longrightarrow \cdots$$

2. For any pair of distinct tasks i and j, no pair of $CS_i^{[k]}$ and $CS_j^{[k']}$ are concurrent. Hence, each task may be written as follows:

   therefore, a task i generates the following sequence:

$$NCS_i^{[1]} \longrightarrow trying_i^{[1]} \longrightarrow CS_i^{[1]} \longrightarrow exit_i^{[1]} \longrightarrow NCS_i^{[2]} \longrightarrow \cdots$$

```
initial declaration;
repeat forever
    noncritical section ;
    trying ;
    critical section ;
    exit ;
end repeat
```

THE MUTUAL EXCLUSION PROBLEM - FORMALIZATION

3. The execution time of a Critical Section is finite.
4. No **deadlock** occurs.
   A deadlock is a condition when one or more tasks are “trying to enter” but none ever does.
5. Every tasks attempting to enter its Critical Section must eventually succeed. (Fairness requirement).
   Rem: the strongest fairness condition is that tasks waiting to enter their CS are kept in a First Come First Served queue.
   This requirement also precludes **livelock** and **starvation.**
6. No assumption can be made with respect to to the relative frequency of tasks entering their CS, such as “taking turns”.

CONCEPTS

Concept	Meaning
Concurrent computing	Multiple interacting computational tasks executing in the same period
Atomic operation	An indivisible sequence of instructions
Critical section	A code section encompassing the access to a shared resource.
Mutual exclusion	A requirement that one task cannot execute in its critical section at the same time as another task using the same shared resource.
Deadlock	A situation where a set of tasks are blocked waiting for each other.
Livelock	A situation where a set of tasks are executing (alive) but cannot proceed doing useful work.
Starvation	A runnable task is overlooked by the scheduler or access control to a shared resource. It is always deferred with respect to other tasks.
Race Condition	Multiple tasks read and write to shared resource and the final result depends on the relative timing of these tasks.

POSSIBLE SOLUTIONS FOR THE MUTUAL EXCLUSION PROBLEM AND APPLICABLE ARCHITECTURES

Solutions:

- interrupt disabling;
- based on shared variables;
- based on inter-tasks messages.

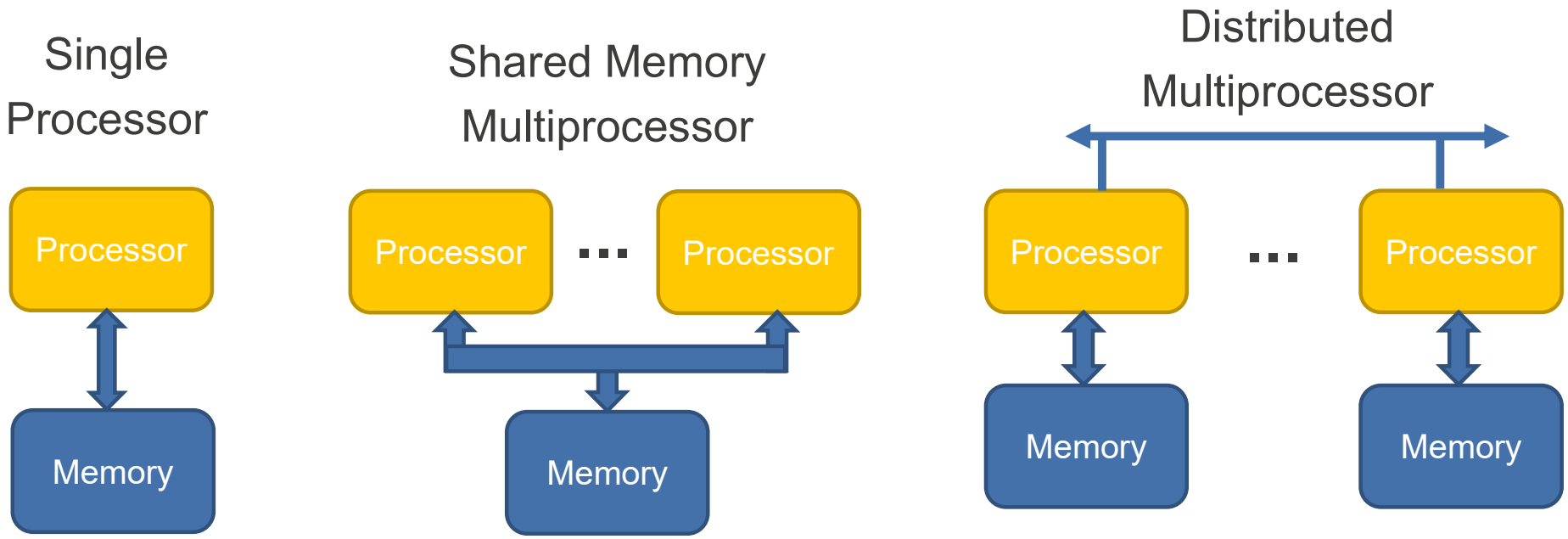
Required support:

- special instruction;
- hardware support;
- none.

Architectures:

- single processor;
- shared memory multiprocessor;
- networked multiprocessor.

TARGET ARCHITECTURES FOR MUTUAL EXCLUSION



SOLUTIONS TO MUTUAL EXCLUSION - INTERRUPT DISABLING

Disabling all interrupts is a simple solution that solves the Mutual Exclusion problem; however, it has the following caveats:

- delays interrupt service even for non-related activities;
- many RTOS do not allow tasks to disable interrupts, as this precludes the correct operation of the RTOS;
- if a tasks “hangs” while interrupt is disabled, the whole system is blocked;
- this solution applies only for the single processor architecture.

SOLUTIONS TO MUTUAL EXCLUSION - SPECIAL INSTRUCTIONS

Special instructions that support Mutual Exclusion are characterized by:

- are atomic (uninterruptable);
- possibly execute a read-modify-write memory cycle without interruption, even in a multiprocessor system.

Examples:

1. test-and-set instruction
2. exchange instruction (e.g. SWAP on ARM7TDMI)

This solution applies to single processor and shared memory multiprocessor.

LDREX AND STREX INSTRUCTIONS - CORTEX-M

```
    MOV     R1, #0x1            ; Initialize the 'lock taken' value
try
    LDREX   R0, [LockAddr]      ; Load the lock value
    CMP     R0, #0              ; Is the lock free?
    ITT     EQ                  ; IT instruction for STREXEQ and CMPEQ
    STREXEQ R0, R1, [LockAddr]  ; Try and claim the lock
    CMPEQ   R0, #0              ; Did this succeed?
    BNE     try                 ; No - try again
    ....                        ; Yes - we have the lock.
```

Syntax

`LDREX{cond} Rt, [Rn {, #offset}]`

`STREX{cond} Rd, Rt, [Rn {, #offset}]`

cond Is an option

Rd Specifies th

Rt Specifies th

Rn Specifies th

offset Is an option
the value in

1. Use a Load-Exclusive instruction to read the value of the location.
2. Modify the value, as required.
3. Use a Store-Exclusive instruction to attempt to write the new value back to the memory location.
4. Test the returned status bit. If this bit is:

   0 The read-modify-write completed successfully.

   1 No write was performed. This indicates that the value returned at step 1 might be out of date. The software must retry the entire read-modify-write sequence.

Software can use the synchronization primitives to implement a semaphores as follows:

1. Use a Load-Exclusive instruction to read from the semaphore address to check whether the semaphore is free.

2. If the semaphore is free, use a Store-Exclusive to write the claim value to the semaphore address.

3. If the returned status bit from step 2 indicates that the Store-Exclusive succeeded then the software has claimed the semaphore. However, if the Store-Exclusive failed, another process might have claimed the semaphore after the software performed step 1.

The Cortex-M4 includes an exclusive access monitor, that tags the fact that the processor has executed a Load-Exclusive instruction. If the processor is part of a multiprocessor system, the system also globally tags the memory locations addressed by exclusive accesses by each processor.

The processor removes its exclusive access tag if:

- It executes a CLREX instruction.
- It executes a Store-Exclusive instruction, regardless of whether the write succeeds.
- An exception occurs. This means the processor can resolve semaphore conflicts between different threads.

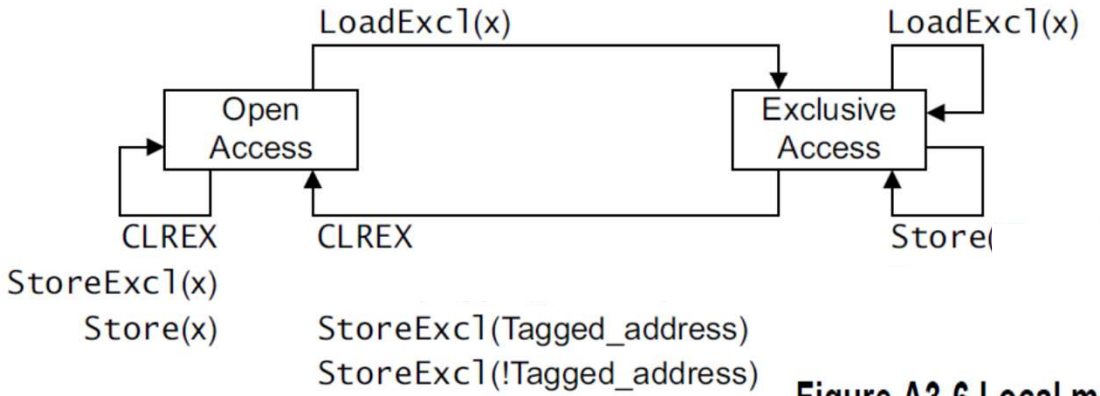


Figure A3-6 Local monitor state machine diagram

In a multiprocessor implementation:

- executing a CLREX instruction removes only the local exclusive access tag for the processor
- executing a Store-Exclusive instruction, or an exception. removes the local exclusive access tags, and global exclusive access tags for the processor.

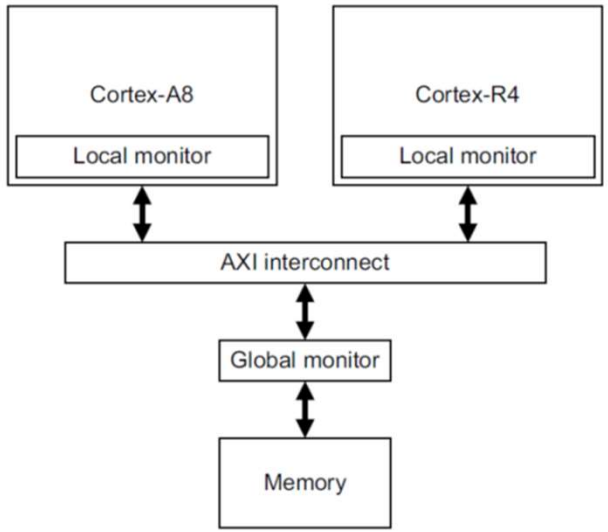


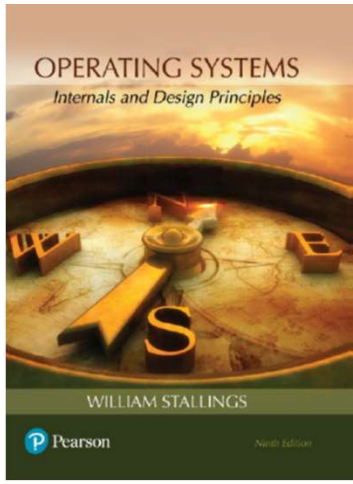
Figure 1-1 Local and global monitors in a multi-core system

SOLUTIONS TO MUTUAL EXCLUSION - SEMAPHORE

```
// Thread A
Inicialization( );
while (1) {
    NonCriticalRegion( );
    if ( MutexGet( ) == SUCESS ) {
        CriticalRegion( );
        MutexPut( );
    }
}
```

```
// Thread B
Inicialization( );
while (1) {
    NonCriticalRegion( );
    if ( MutexGet( ) == SUCESS ) {
        CriticalRegion( );
        MutexPut( );
    }
}
```

SOLUTIONS TO MUTUAL EXCLUSION - SW SOLUTIONS




```
   /* PROCESS 0 /*                  /* PROCESS 1 *
.                                .
.                                .
while (turn != 0)                while (turn != 1)
   /* do nothing */ ;               /* do nothing */;
/* critical section*/;            /* critical section*/;
turn = 1;                        turn = 0;
.                                .
```

(a) First attempt

Alternating tasks

SOLUTIONS TO MUTUAL EXCLUSION - SW SOLUTIONS

```
   /* PROCESS 0 *          /* PROCESS 1 *
.                          .
.                          .
while (flag[1])            while (flag[0])
   /* do nothing */;          /* do nothing */;
flag[0] = true;            flag[1] = true;
/*critical section*/;      /* critical section*/;
flag[0] = false;           flag[1] = false;
.                          .
```

(b) Second attempt

```
   /* PROCESS 0 *          /* PROCESS 1 *
.                          .
.                          .
flag[0] = true;            flag[1] = true;
while (flag[1])            while (flag[0])
   /* do nothing */;          /* do nothing */;
/* critical section*/;      /* critical section*/;
flag[0] = false;           flag[1] = false;
.                          .
```

(c) Third attempt

No mutual exclusion

Deadlock

SOLUTIONS TO MUTUAL EXCLUSION - SW SOLUTIONS

```
   /* PROCESS 0 *                /* PROCESS 1 *
.                             .
.                             .
flag[0] = true;               flag[1] = true;
while (flag[1]) {             while (flag[0]) {
   flag[0] = false;              flag[1] = false;
   /*delay */;                  /*delay */;
   flag[0] = true;               flag[1] = true;
}                             }
/*critical section*/;         /* critical section*/;
flag[0] = false;              flag[1] = false;
.
```

(d) Fourth attempt

Possible Livelock
when tasks synchronize access to flags

SOLUTIONS TO MUTUAL EXCLUSION - SW SOLUTIONS

Peterson's Algorithm

Use shared data, hence, can be used in single-processor and shared memory multiprocessor.

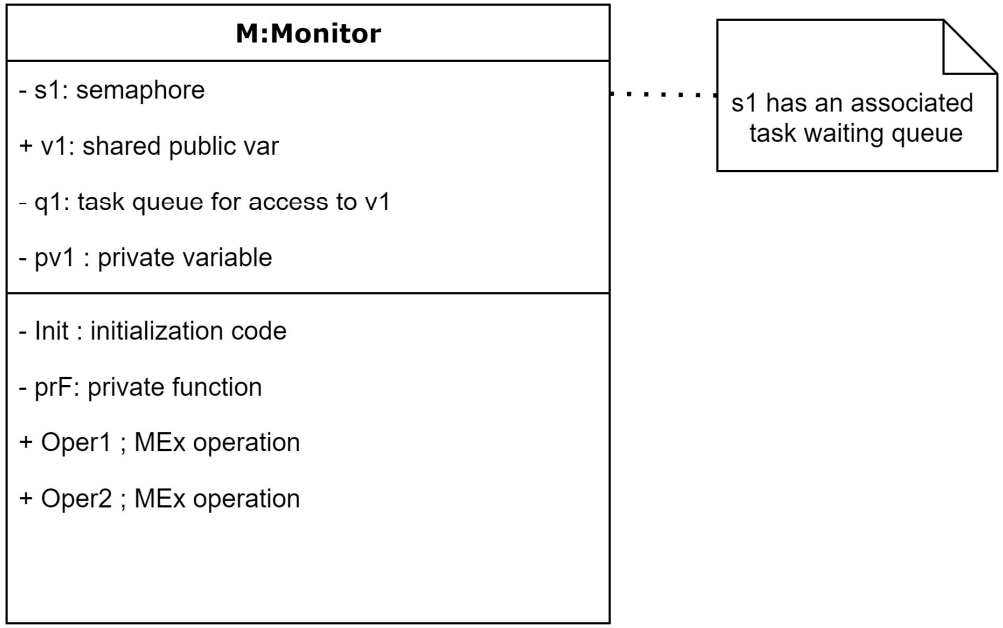
```
boolean flag [2];
int turn;
void P0()
{
      while (true) {
            flag [0] = true;
            turn = 1;
            while (flag [1] && turn == 1) /* do nothing */;
            /* critical section */;
            flag [0] = false;
            /* remainder */;
      }
}
 void P1()
{
      while (true) {
            flag [1] = true;
            turn = 0;
            while (flag [0] && turn == 0) /* do nothing */;
            /* critical section */;
            flag [1] = false;
            /* remainder */
      }
}
void main()
{
      flag [0] = false;
      flag [1] = false;
      parbegin (P0, P1);
}
```

MONITOR

OOP - concept of ENCAPSULATION

All routines accessing a shared resource are located in a single SW module !

Access to these routines is protected by (e.g.) semaphores.

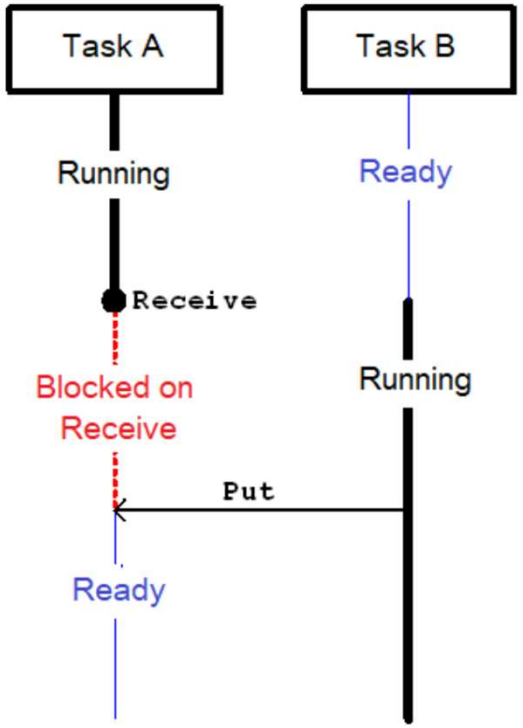


SOLUTIONS TO MUTUAL EXCLUSION - MESSAGE BASED SOLUTIONS

So far, the solutions presented require the use of shared variables, hence, these are not applicable to a Distributed architecture.

Message based solutions may be used in any of the three architectures being considered.

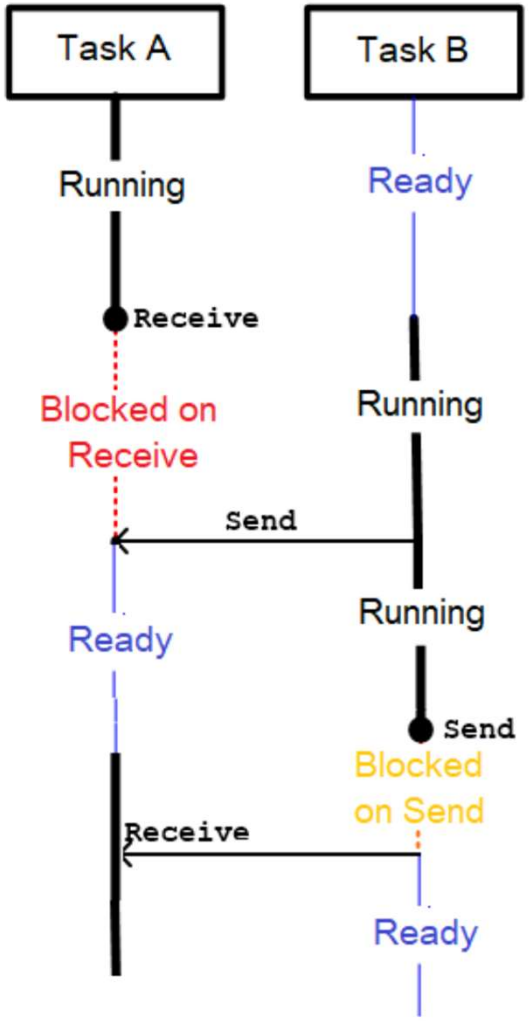
ASYNCHRONOUS COMMUNICATION



SYNCHRONOUS COMMUNICATION

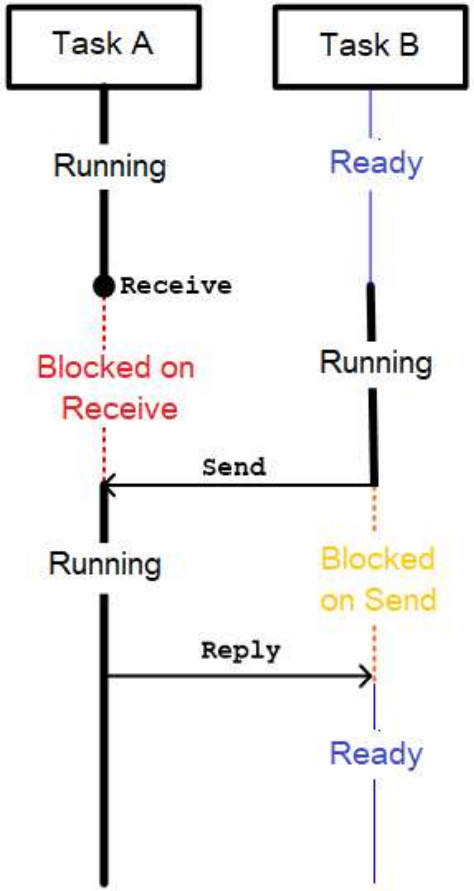
Message exchange only occurs when both tasks are willing to communicate.

Data can be transferred directly from sender to receiver.



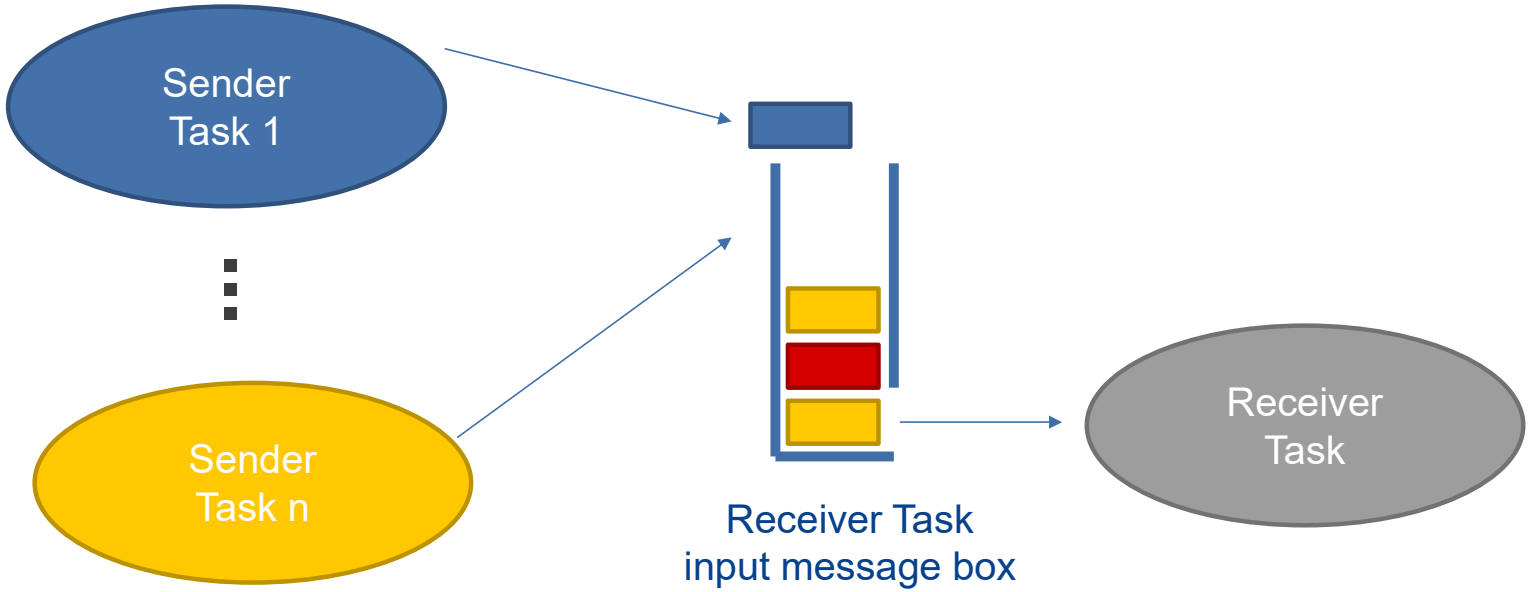
SYNCHRONOUS COMMUNICATION WITH REPLY (SEND-RECEIVE-REPLY)

Send and Receive operations are blocking.



SOLUTIONS TO MUTUAL EXCLUSION - MESSAGE BASED SOLUTIONS

Messages as a sequentialization solution to Mutual Exclusion.



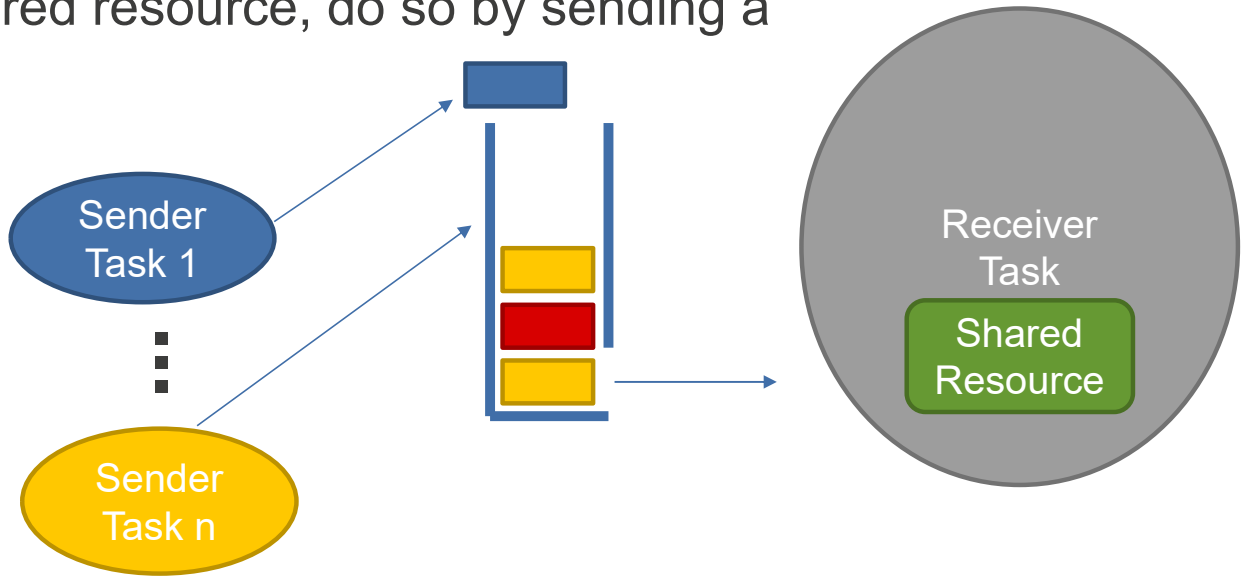
MESSAGE BASED SOLUTION

Evolving the Monitor concept to a Client-Server model.

Server encapsulates the shared resource.

Tasks willing to use the shared resource, do so by sending a message.

Server replies with result of operation.



`Semaphore`	An integer value used for signaling among processes. Only three operations may be performed on a semaphore, all of which are atomic: initialize, decrement, and increment. The decrement operation may result in the blocking of a process, and the increment operation may result in the unblocking of a process. Also known as a **counting semaphore** or a **general semaphore**
Binary Semaphore	A semaphore that takes on only the values 0 and 1.
Mutex	Similar to a binary semaphore. A key difference between the two is that the process that locks the mutex (sets the value to zero) must be the one to unlock it (sets the value to 1).
Condition Variable	A data type that is used to block a process or thread until a particular condition is true.
Monitor	A programming language construct that encapsulates variables, access procedures and initialization code within an abstract data type. The monitor's variable may only be accessed via its access procedures and only one process may be actively accessing the monitor at any one time. The access procedures are *critical sections*. A monitor may have a queue of processes that are waiting to access it.
Event Flags	A memory word used as a synchronization mechanism. Application code may associate a different event with each bit in a flag. A thread can wait for either a single event or a combination of events by checking one or multiple bits in the corresponding flag. The thread is blocked until all of the required bits are set (AND) or until at least one of the bits is set (OR).
Mailboxes/Messages	A means for two processes to exchange information and that may be used for synchronization.
Spinlocks	Mutual exclusion mechanism in which a process executes in an infinite loop waiting for the value of a lock variable to indicate availability.

